Lead — reviewing and merging

The lead's main job: **nothing reaches `dev` or `main` without the lead's review.**

```
main   ← only the lead merges dev into it, when dev is tested and stable
 └ dev ← every pull request goes here, after the lead's review
    └ feature/…, fix/… ← each person's branch, made from dev
```

Who does what

Person	Builds, in order
Havindu	B-3 Achievements → B-5 Feedback → B-9 Year-end
Mansala	B-6 Complaints → B-8 Permission cards → B-4 Certificates
Apeksha	B-11 Profile details → B-12 Smaller ones → B-13 Settings → B-10 Dashboards
Lead	B-7 Notifications · every review

Order between people: B-4 waits for B-3 · B-9 needs B-13's School Year dates · B-10 and B-7 go last.

One-time GitHub set-up

1. **Add the three as collaborators:** repo → Settings → Collaborators → Add people (their GitHub usernames). Or: `gh api -X PUT repos/SDJ-Ro/mock/collaborators/<username> -f permission=push`
2. **Branch protection on `main` and `dev`** (already set — check it in Settings → Branches): pull request required, 1 approval, code owner review (`.github/CODEOWNERS` = you), the `check` job must pass, no force-push, no deleting. `dev` is the default branch, so new pull requests point at it.
3. **Merge button:** Settings → General → Pull Requests → allow **merge commits**; untick squash and rebase (every small commit and its author stay in the history). Tick "Automatically delete head branches".

Reviewing a pull request

```
gh pr list                      # what is waiting
gh pr checkout 12               # get the branch locally
bash bin/check.sh               # 0 errors
php database/update.php         # if it adds an update
```

Then in Claude Code: "use rules-reviewer on this branch", and open every page it touches as every role that sees it.

Send it back when:

- it changes files outside its task, or the look of a page;
- `bin/check.sh` has an error, or the GitHub check is red;
- SQL outside a model, `$_POST`, no `e()`, a typed role list, a typed number that belongs in `rules.php`, a typed school name or word, inline `style=""`, a new library;
- the example data it replaces is still there, or dead code is left behind;
- the curl attacks weren't done, or the template's "Still open" is empty when something clearly is;

- the author can't explain a part of it when you ask (ask!).

Comment on the exact line, "Request changes", and say what to do. Approve only what you would defend in the viva yourself.

Merging

"Create a merge commit". Then: pull `dev`, run `bin/check.sh`, open the pages once more.

`dev` → **`main`** (after a task is finished and `dev` works end to end):

```
gh pr create --base main --head dev --title "Release: B-5 Feedback" --fill
```

Check it the same way, then merge it yourself.

Keeping it on track

- Ask for a **draft** pull request on day 1 of each task (the plan), so you see the direction before the code.
- Small pull requests: one task step each. A 2,000-line pull request goes back to be split.
- A daily 10-minute check: what's merged, what's blocked, who is waiting for whom.